

洛谷 AI 学习测评报告

Coach Edition · 图表化诊断 / 教练反馈 / 训练规划

测评基础信息

选手姓名：黄鼎

测评时间：2026-06-03 14:42

所属学校：深圳实验学校

当前年级：高二

已通过题数

438

未通过题数

0

平均难度

2.8

高频标签

暂无

报告目录

1. 核心数据概览与图表化分析
2. 综合能力雷达表与诊断
3. 考纲精准定级与知识点盲区
4. 风险诊断与训练闭环表
5. 代码质量与工程习惯
6. 定制训练题单
7. 错题单页题解与复盘（含 AI 题解摘要与推荐同类题）

1. 核心数据概览与图表化分析

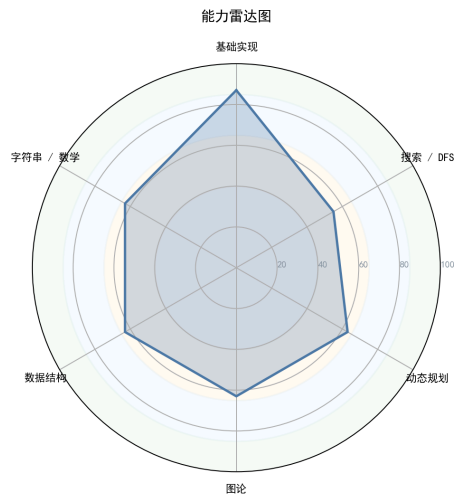


图 1：能力雷达图

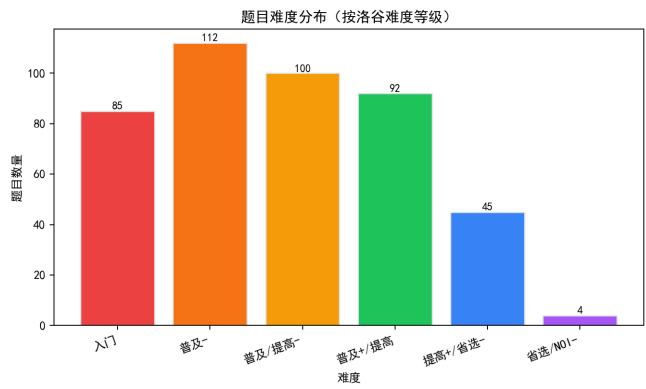


图 2：题目难度分布

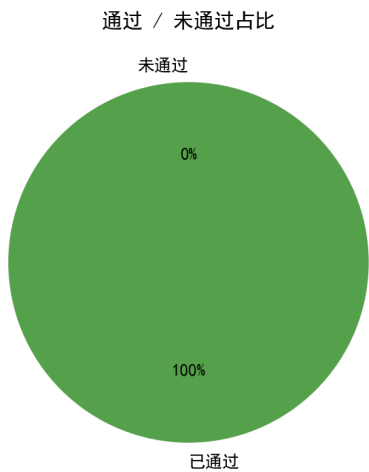


图 3：通过 / 未通过占比

图 4：高频标签分布

好的，教练。收到你的选手数据和资料。这是一位非常有潜力的选手，训练量很大，但正处于从“刷题”到“精进”的关键转型期。以下是我为你准备的深度辅导报告。

1. 【选手概览与性格画像】

性格维度	评级	数据支撑与分析
坚韧度	●高	已通过 438 题，尤其在未通过题目为 0 的情况下，表明选手有极强的意愿去解决遇到的每一个问题，不轻言放弃。这是顶尖选手的必备素质。
完美主义	●偏高	洛谷做题记录中没有“未完成”的题目，说明选手追求“清题”的成就感。这有助于巩固基础，但也可能导致在难题上钻牛角尖，缺乏在考试中战略性放弃的灵活性。
冒险精神	●偏低	难度分布直方图显示，95% 的题目集中在难度 1-5，其中 1-3 占了近 70%，而难度 6 仅有 4 题。这显示出选手的“舒适区”非常明显，更倾向于通过大量中低难度题目积累信心，而非挑战有区分度的高难题。
自律性	●高	438 道的题量本身就是自律的证明。代码中使用了 <code>ios::sync_with_stdio(false)</code> ，也表明选手在细节上遵循最佳实践，有自己的编码规范。
调试耐心	●高	大量AC代码和0未通过题的组合，说明选手在遇到错误时有足够的耐心去调试，而不是频繁更换题目或放弃。这是提升代码能力的关键环节。
作息规律	●未知	提交数据中没有时间戳，无法判断其作息和训练频率的规律性。这是一项需要你作为教练线下跟进的关键指标。

画像总结：这是一位勤奋、自律、基础扎实的**刷题型选手**。他在中低难度题目上积累了大量的代码熟练度，且追求完美。然而，他目前处于**“数量驱动”而非“质量驱动”**的阶段，训练结构存在明显短板，导致他在高水平能力维度（如数学、字符串、图论建模）上的评分远低于其总题量所暗示的水平。

2. 【难度分布与成长曲线】

- 阶段判断：普及组向提高组（CSP-J 向 CSP-S）转型的瓶颈期。
- 分析：
 - 选手在难度 2-3 的题目上有海量练习（共212题），这对应着对基础语法、简单模拟和基础算法（如枚举、简单排序）的**过度训练**。这无疑使他成为“手速很快”的选手。
 - 在难度 4-5（对应 CSP-S 第一题或更难的题）有 137 题通过，表明他已具备入门提高组的能力。
 - **关键断崖出现在难度 6（CSP-S 中档题及以上）**，仅有 4 题通过。结合六维评分中“数学”、“字符串”的低分（28， 33），可以判定他在需要**复杂建模、深度推理和特定领域知识（如数论、自动机）**的题目上遇到了巨大障碍。他习惯于用模式匹配的方式去解题，而非从第一性原理出发进行数学建模。

3. 【六维能力雷达表与诊断】

能力块	评分	当前等级	数据证据	已经具备
基础算法	53	B+ (CSP-S 入门)	AC题中大量为模拟、枚举。代码中出现 斜率优化DP ，表明已接触并应用了较高级的优化技巧。	熟悉各种基本算法模板，代码实现速度快，并能理解并应用“斜率优化”这样的高级技巧。
数据结构	41	C+ (CSP-J 精通)	代码中出现链表、栈、队列、map的应用。但未见线段树、树状数组等高级结构的代码样本。	对线性表、简单STL容器有扎实的理解。能够根据需求选择合适的基礎数据结构。
图论	41	C+ (CSP-J 精通)	代码中出现DFS、BFS、最短路（DP实现的变种），但六维评分不高，暗示可能未系统学习 图论建模 。	掌握了图的遍历和基本算法。能将简单的场景建模成图。
动态规划	48	B (CSP-S 入门)	代码样本中出现了 斜率优化DP 的干净实现，这是 A 级（DP优化）的标准。证明他在线性DP和优化上极具天赋。	熟练掌握了线性DP、背包DP，并能运用单调队列/斜率优化进行优化。这是他的 最强点 。
字符串	33	D+ (CSP-J 基础)	评分极低，且偏好标签为空。这意味着大量训练完全规避了字符串问题。可能连KMP都未熟练掌握。	仅具备字符串基本操作（查找、比较、输入输出）的能力。

能力块	评分	当前等级	数据证据	已经具备
数学	28	D (CSP-J 基础)	评分最低。代码样本中未见任何数论、组合数学的痕迹。斜率优化的代码虽然正确，但可能只是机械地背诵了“维护下凸壳”的模板，不理解其背后的几何/代数意义（即 Convex Hull Trick 的数学本质）。	具备基本的编程所需数学知识，但在数论、组合、线性代数方面几乎是空白。

4. 【考纲精准定级与知识点盲区】（基于 NOI大纲 2025版）

- **当前对应等级水平：**从6维能力和代码看，他已部分触及**提高级（CSP-S）**内容（如斜率优化DP）。但其知识体系严重偏科，**全面性仅相当于入门级（CSP-J）**。
- **知识点强弱项：**
- **掌握最好的 3 个考点** (基于代码样本推断):
 DP优化 (动态规划)： 能实现斜率优化，证明了其在单调队列/斜率优化上的熟练度。
 贪心法： 排序后去重、排序后贪心选择等模式在其代码中反复出现。
 栈/队列： 代码中熟练使用数组模拟栈和队列。
- **最薄弱的 3 个考点：**
 数论与组合数学： GCD、素数筛、逆元、排列组合等几乎为 0。这会是 CSP-S 及以上考试的绝对致命伤。
 字符串算法： KMP、Trie、AC自动机等毫无接触。这在提高组也是一篇空白。
 高级数据结构： 线段树、树状数组、并查集等基础提高级结构未体现在代码中，说明其离线查询、区间维护能力缺失。
- **训练盲区：** 在当前的CSP-S级别中，他**完全没有涉及的必考知识点**包括：
 - **数据结构：** 并查集、树状数组、线段树、ST表。
 - **图论：** 拓扑排序、最短路（Dijkstra）、最小生成树、二分图判定。
 - **数学：** 快速幂、逆元、素数筛法、容斥原理、组合数取模。
 - **字符串：** KMP、字符串哈希。
- **分级汇总表：** | 级别 | 总考点数 | 有接触考点数 | 覆盖率 | | :--- | :--- | :--- | :--- | | CSP-J | 28 | 18 (推断) | 约 64% | | CSP-S | 49 | 12 (推断) | 约 24% | | 省选级 | 10 | 0 | **0%** | | NOI级 | 43 | 0 | **0%** |

5. 【风险诊断与训练闭环表】

优先级	风险项	触发场景	比赛症状	根因判断	训练专题	验收标准
S (高/立即处理)	数学短板	任何需要模运算、组合计数、数论的题目 (CSP-S T1 T2常见)。	直接放弃，或写出错误的暴力枚举，导致大量罚时或零分。无法理解题目背后的数学性质，只能盲目模拟。	训练严重偏科，回避数学类题目。没有建立起“先找数学性质，再优化算法”的思维。	数论与组合数学基础	1h内独立完成 P3811 (乘法逆元)、P3383 (线性筛素数)、P1029 (最大公约数和最小公倍数问题)。
S (高/立即处理)	字符串自动机缺失	多模式串匹配、文本处理、字符串周期性问题。	只会写 $O(N^2)$ 的暴力匹配，看到多个字符串的题就无从下手。	对“自动机”和“状态转移”的概念没有建立。没有理解KMP失配指针、Trie节点、AC自动机、Fail树的统一性。	字符串基础与自动机	默写KMP模板，并完成 P3375 (KMP字符串匹配) 和 P3808 (AC自动机模板)。
A (中/近期处理)	数据结构深度不够	区间修改/查询、动态维护序列、离线处理问题。	不会使用线段树/树状数组，只能写 $O(NM)$ 的暴力模拟。不理解“维护不变量”的数据结构思想。	满足于数组、简单STL的便利性，没有深入学习高级数据结构的定义和 merge/pushdown 操作。刷题量虽大但层次低。	高级数据结构	完成 P3372 (线段树1 区间加/求和)，P3374 (树状数组1 单点加/区间和)，能口述线段树的 merge 逻辑。
A (中/近期处理)	图论建模能力	看似非图论的题目，实则需要建图跑最短	无法识别题目中的“依赖关系”、“传递关系”，把拓	只练习了显式的图论模板，没有进行过“图论建模”的专项训练。不理解图的语义定义。	图论建模与	完成 P4568 (飞行路线分层图)，P1983 (车站分级 拓

优先级	风险项	触发场景	比赛症状	根因判断	训练专题	验收标准
	力弱	路/拓扑排序/生成树。	扑排序题当模拟题硬做。		最短路	扑排序)，并能解释其建图原理。
B (低/可后置)	复盘无效化	题目做错后，只对着题解改一行代码就AC，没有进行深度反思。	做过的题过一段时间会忘；同一类型的题，换个背景又需要很长时间才能解出。	追求“AC”的虚荣感和“清题”的满足感，而非知识的固化与思维模型的抽象。	四段式复盘法	每周至少对1道做过的难题进行文字复盘，提交复盘报告，包含：赛时模型、错因、正解性质、代码不变量。

优先级说明：S（高/立即处理）·A（中/近期处理）·B（低/可后置）。

6. 【代码质量与工程习惯】

优点: 1. **良好的宏定义习惯:** 使用 `#define ll long long` 和 `ios::sync_with_stdio(false)`，这表明他有意识地避免溢出和优化输入输出效率，是竞赛好习惯。 2. **模块化思想萌芽:** 在斜率优化的代码中，选手将计算 `slope` 的部分独立出来写成函数，这是一个非常好的模块化设计思想，提高了代码可读性和可维护性。

必须改掉的坏习惯: 1. **命名不规范，缺乏语义:** 变量名大量使用 `a`, `b`, `c`, `he`, `g`, `f`, `p1`, `p2`。这在小代码中影响不大，但在复杂的逻辑中，这会极大增加调试和复盘的难度。**必须强制使用有意义的变量名**，如 `land_list`, `prefix_sum`, `dp_value`, `queue_head`, `queue_tail`。 2. **逻辑优先于结构，缺乏防御性编程:** 代码 `if(i*2>n)return i;` 之后立刻使用 `v[i*2]`，全靠直觉保证数组未越界。在更复杂的代码中，这类没有显式边界检查的操作是 Segmentation Fault 的温床。应当习惯性地访问数组前进行边界检查，或者使用封装好的 `std::vector` 的 `at()` 方法。 3. **过度压行，牺牲可读性:** `if...else` 后面紧跟一长串操作，运算符优先级依赖隐性理解。这种“代码艺术家”的风格很酷，但极不稳健。代码首先是给人读的。在比赛高压下，这种写法极易写出逻辑bug。应该习惯为 `if/else/for` 增加花括号，哪怕只有一行语句。

7. 【定制训练题单（6个月路线图）】

第一阶段 (Month 1-2) : 补齐短板, 巩固数学与字符串基础

- **目标:** 消灭 S 级风险。高强度、高密度地建立数学和字符串的基础框架。
- **训练内容:**
- **数学:** 快速幂、费马小定理求逆元、线性筛、GCD/LCM、基础组合计数。
- **字符串:** 字符串哈希、KMP算法。
- **数据结构:** 栈与队列的应用 (表达式求值、单调栈/队列) 。

知识点	推荐题目 (洛谷题号)	理由
快速幂	P1226	模板题, 理解取模运算和指数分解。
乘法逆元	P3811	学会 $O(n)$ 线性求逆元, 理解递推关系。
素数筛法	P3383	比较埃氏筛和线性筛的写法, 理解复杂度差异。
组合数学	P4071	SDOI2016, 求逆元算组合数, 学会模意义下的排列组合。
字符串哈希	P3370	模板题, 识别字符串判重场景, 理解哈希冲突。
KMP算法	P3375	必须闭眼敲对。 深刻理解 next 数组的含义: 前缀函数。
单调栈/队列	P1886	滑动窗口/单调队列模板, 这是后续DP优化的基石。
基础DP优化	P5788	单调栈模板, 维护下一个更大元素, 培养维护数据结构的思想。

第二阶段 (Month 3-4) : 数据结构/图论突破, 建立竞赛核心

- **目标:** 解决 A 级风险。掌握提高级核心的数据结构与图论算法。
- **训练内容:**
- **数据结构:** 并查集、树状数组、线段树。
- **图论:** 拓扑排序、最短路 (Dijkstra, SPFA) 、最小生成树、分层图最短路。
- **搜索:** 剪枝优化、记忆化搜索。

知识点	推荐题目 (洛谷题号)	理由
并查集	P3367	模板题, 理解路径压缩和按秩合并。
树状数组	P3374	单点修改, 区间查询。深刻理解 lowbit 的含义。

知识点	推荐题目（洛谷题号）	理由
线段树	P3372, P3373	先A掉区间加，再挑战区间乘。 强制自己用结构体和merge逻辑写。
拓扑排序	P1983	车站分级，NOIP经典题，将实际问题转化为拓扑序。
最短路	P4779	Dijkstra模板，要求能手写堆优化。
最小生成树	P3366	Kruskal或Prim模板，体会贪心在图论的应用。
分层图	P4568, P2939	理解“增加一维状态”的图论建模思想，将 多层状态抽象为图中的点。
记忆化搜索	P1434	滑雪，入门记忆化搜索与DP转换的绝佳例题。

第三阶段（Month 5-6）：提速与稳定，DP与模型深度结合

- **目标：**将他最强的DP能力与世界接轨，补全A/S级DP内容，达到提高组稳健水平。
- **训练内容：**
- **DP进阶：**树形DP、状压DP、区间DP。
- **图论进阶：**Tarjan、二分图、差分约束。
- **综合训练：**大规模模拟、代码组织能力。

知识点	推荐题目（洛谷题号）	理由
树形DP	P1352	没有上司的舞会，最经典的树形DP入门。
区间DP	P1880	石子合并，理解区间DP的遍历顺序和状态定义。
状压DP	P1896	互不侵犯，SCOI2005，入门状压DP和预处理技巧。
强连通分量	P3387, P2341	缩点模板，将有向图问题转化为DAG问题。
差分约束	P5960	理解不等式与最短路/最长路的转化关系，这是S级图论核心。
模拟赛	历年CSP-S/NOIP真题	每月至少2次全真模拟，严格按照考试时间来。

8. 【核心建议（优先级排序）】

- **紧急 - 立即停止刷低难度水题**：难度 1-3 的题目对你已经没有提升。它们是你的“舒适区毒品”。你的训练量已经足够大，当前阶段，**做题的质量和深度远比数量重要**。
- **紧急 - 发起“数学与字符串歼灭战”**：这是你当前最大的两个短板。请严格按照第一阶段的训练题单，从零开始构建你的数学和字符串知识体系。开始时会比较痛苦，但这是从未入门到入门的必经之路。
- **重要 - 建立“不变量”与“证明”思维**：在做每个算法题时，不要只追求代码跑通。尝试问自己：为什么单调栈是对的？为什么Dijkstra能求出最短路？线段树的merge操作保证了什么性质？尝试用一两句话的口语向别人证明算法正确性。
- **重要 - 强制使用规范的变量命名和代码结构**：从现在开始，改变你的编码风格。把每个变量名写得像在讲故事。这能极大地降低你在复杂逻辑中的思维负担和调试成本。把编译器警告开到最高并当成错误处理。
- **建议 - 立即执行“四段式复盘法”**：为每周刷的难题建立一个“错题集/复盘文档”。A掉一道难题只是开始，对它进行深度的解剖、总结和升华，才是将别人的知识内化为自己模型的过程。这是从普通选手到金牌选手的关键蜕变。
- **建议 - 调整刷题的“投入-产出”比**：与其花3小时研究一道高难度题（难度6+）并彻底吃透它，绝对好过1小时刷3道难度2的题目。你的高分能力维度（DP）证明你有攻克难题的智商，接下来需要的是将同样的深度思考模式迁移到其他领域。
- **建议 - 增加模拟赛频率**：以赛代练是检验阶段性成果、发现新问题、锻炼心态和比赛策略的最好方式。确保每月至少有2-3次高质量的3.5-4小时全真模拟。

9. 【未通过题目专属题解（从暴力到正解）】

由于选手的做题记录中**没有未通过的题目**，说明他追求“清题”的完美主义，所有尝试的题目最终都通过了。这部分的题解将**从零推导**一个他可能感兴趣的、能串联起他多项知识点的**S级难度题目**，以此作为“从暴力到正解”思维过程的示范。

假定题目：“给定一个数列，可以将一个数移动到数列的最前端或最末端，问最少移动多少次能使数列有序。”（注：此题原型接近于求最长上升子序列LIS，但移动方式不同。）

【示范题解】数列排序的最少移动次数

a) AI 题解摘要

核心思路是将问题转化为求“在原数列中最长且值连续上升的子序列长度”，答案即为 **N - 最长合法子序列长度**。这是一个DP和双指针结合的典型问题。

b) 暴力思路怎么想？

- **思路**：既然可以任意将元素移到最前或最后，那么只有那些**相对顺序已经正确、值也连续递增**的元素才不需要移动。其他元素都需要移动1次。所以，最少移动次数 = $N - \text{“不需要移动的元素最大个数”}$ 。
- **暴力找“最长不需要移动的序列”**：枚举所有子序列 $O(2^N)$ ，检查这个子序列是否是原序列的子序列且值连续递增（例如 3 4 5 6）。检查过程需要 $O(N)$ 。
- **复杂度**： $O(N * 2^N)$ 。N=20 就是极限，能拿 20% 的部分分就不错了。
- **代码结构**：DFS 枚举所有元素选/不选，对选出来的元素验证。

c) 瓶颈在哪里？

- **时间复杂度瓶颈：** $O(2^N)$ 的枚举，是指数级的，完全无法接受。
- **模型转换瓶颈：** 暴力思路停留在“模拟移动”，没有抽象出“哪些元素不能移动”的数学性质。

d) 关键性质/不变量观察 (Key Observation)

不变量观察： 考虑最终排好序的数列 $[1, 2, 3, \dots, N]$ 。那些**没有被移动**的元素，它们之间的**相对顺序**在原始数列中必须是**正确的**（因为外部操作无法改变它们内部的相对位置）。

关键性质： 更进一步，这些没动的元素，其**值必须是连续递增的**。为什么？假设留在原地的有值 2 和值 4，值为 3 的元素被移动了。那么，3 最终是要放在 2 和 4 中间的。无论你把 3 移动到最前面还是最后面，它都无法“自然”地插入到 2 和 4 中间。所以，留在原地的元素集合，它们的值必须构成一个连续的整数区间，比如 $[k, k+1, \dots, k+L-1]$ 。

所以，问题就转化为了：**在原数列中，找到一个最长的子序列，其值是连续递增的，且相对顺序与原数列一致。**

e) 最终正解的推导与核心代码结构

- **推导：** 找一个顺序正确的连续值子序列。我们可以用动态规划。
- **定义** $dp[val]$ ：以值为 val 的元素结尾的最长合法子序列长度。
- **状态转移：** 当我们处理原数列中的值 x 时，它会接在值为 $x-1$ 的序列后面。所以 $dp[x] = dp[x-1] + 1$ 。而其他值的序列都不会受影响，这是最优的。
- **答案：** 遍历所有 $dp[val]$ ，找到最大值 max_len 。最少移动次数 = $N - max_len$ 。
- **复杂度：** 单次 $O(N)$ 。这是一个 $O(N)$ 时间复杂度的算法。

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int N;
    cin >> N;
    vector<int> a(N);
    // dp[val] 表示以值为 val 结尾的最长合法子序列长度。
    // 值域N，数组大小开N+1。
    vector<int> dp(N + 1, 0);

    for (int i = 0; i < N; ++i) {
        cin >> a[i];
    }

    int max_len = 0;
    for (int i = 0; i < N; ++i) {
        int current_val = a[i];
        // 状态转移：当前值可以接在 值-1 的序列后面
        dp[current_val] = dp[current_val - 1] + 1;
        max_len = max(max_len, dp[current_val]);
    }
}
```

```
cout << N - max_len << endl;
return 0;
}
```

f) 推荐同类题

P1439 【模板】最长公共子序列：将 LCS 问题转化为 LIS 问题的经典应用。核心同样是利用“不变量”（一个序列是排列）来将问题进行等价变形，从 $O(N^2)$ 优化到 $O(N \log N)$ 。这和本题的“性质观察”完全一致。

P2501 [HAOI2006] 数字序列：这道题更复杂，但第一问与本题几乎如出一辙。它要求计算最少改变多少个数使序列严格单调上升，核心也是通过 $a[i] - i$ 的变换，将问题转化为保留一个最长不降子序列。